

Indeksy

Przemysław Starosta - 2025



Explain vs Analyze

Różnica między EXPLAIN PLAN a EXPLAIN ANALYZE w PostgreSQL polega na tym, że EXPLAIN PLAN (lub po prostu EXPLAIN) **pokazuje *przewidywany plan*** wykonania zapytania, natomiast EXPLAIN ANALYZE *wykonuje* zapytanie i **pokazuje *rzeczywisty plan*** wykonania wraz ze statystykami czasu wykonania



Explain vs Anlyze

EXPLAIN (PLAN)

- **Cel:** Pokazuje, jak PostgreSQL *zamierza* wykonać zapytanie, bez faktycznego jego uruchamiania.
- **Wynik:** Zawiera informacje o kolejności operacji (np. skanowanie tabel, łączenia, sortowanie), typach użytych indeksów, szacowanych kosztach (czas, liczba wierszy) dla każdej operacji.
- **Zastosowanie:** Przydatne do szybkiej oceny potencjalnych problemów z wydajnością bez wpływu na dane (ponieważ zapytanie nie jest wykonywane). Można go użyć do sprawdzenia, czy optymalizator wybiera sensowny plan.
- **Brak wykonania:** Zapytanie nie jest faktycznie uruchamiane, więc nie ma wpływu na dane ani nie zużywa rzeczywistych zasobów poza generowaniem planu.



Explain vs Analize

EXPLAIN ANALYZE

- **Cel:** Wykonuje zapytanie i pokazuje *rzeczywisty* plan wykonania wraz ze statystykami czasu wykonania dla każdej operacji.
- **Wynik:** Oprócz informacji z EXPLAIN, EXPLAIN ANALYZE dodaje rzeczywiste czasy wykonania (w milisekundach), liczbę zwróconych wierszy, liczbę pętli dla każdej operacji.
- **Zastosowanie:** Niezbędne do dokładnej analizy wydajności. Pozwala porównać szacowane koszty z rzeczywistymi, co pomaga zidentyfikować, gdzie optymalizator mógł się pomylić w swoich szacunkach (np. z powodu nieaktualnych statystyk). Pomaga również zlokalizować wąskie gardła w zapytaniu.
- **Wykonanie zapytania:** Zapytanie jest faktycznie uruchamiane, więc ma wpływ na dane (jeśli jest to zapytanie modyfikujące dane, np. INSERT, UPDATE, DELETE) i zużywa rzeczywiste zasoby. Z tego powodu należy używać go ostrożnie na środowiskach produkcyjnych, szczególnie w przypadku długo trwających zapytań.



Typy indeksów - B-tree

B-tree (Balanced Tree)

- To domyślny i najczęściej używany typ indeksu w PostgreSQL. Jest to indeks ogólnego przeznaczenia, który doskonale sprawdza się w przypadku:
- **Porównań równościowych:** WHERE column = 'value'
- **Porównań zakresowych:** WHERE column > 'value', WHERE column BETWEEN 'a' AND 'b'
- **Sortowania:** ORDER BY column
- **Wzorców LIKE:** WHERE column LIKE 'prefix%' (ale nie '%suffix')



Typy indeksów - Hash

Hash

- Indeksy hash są używane tylko do porównań równościowych. Są one zazwyczaj szybsze niż B-tree dla zapytań równościowych, ale mają pewne ograniczenia:
- **Brak wsparcia dla zakresów:** Nie można ich używać do zapytań typu WHERE column > 'value'.
- **Brak wsparcia dla sortowania:** Nie pomagają w sortowaniu danych.
- **Brak wsparcia dla wzorców LIKE.**
- **Brak wsparcia dla unikalności:** Nie mogą wymuszać unikalności wartości.



Typy indeksów - GIN

GIN (Generalized Inverted Index)

- GIN to indeks odwrócony, który jest bardzo efektywny w przypadku danych zawierających wiele wartości w jednej kolumnie (np. tablice, dokumenty JSONB, typy pełnotekstowe). Jest idealny do:
- **Wyszukiwania pełnotekstowego:** Bardzo efektywny dla tsvector i operatorów wyszukiwania pełnotekstowego.
- **JSONB:** Wyszukiwanie kluczy i wartości w dokumentach JSONB.
- **Tablice:** Wyszukiwanie elementów w tablicach.



Typy indeksów - BRIN

BRIN (Block Range Index)

- BRIN to indeks zakresowy bloków, który jest przeznaczony dla bardzo dużych tabel, gdzie dane są fizycznie uporządkowane w sposób skorelowany z wartościami kolumny.
- Zamiast indeksować każdą pojedynczą wartość, BRIN przechowuje minimalne i maksymalne wartości dla zakresów bloków danych. Jest to bardzo kompaktowy indeks, ale efektywny tylko wtedy, gdy dane są dobrze uporządkowane.
- **Duże tabele z uporządkowanymi danymi:** Np. tabele z danymi czasowymi, gdzie nowe rekordy są zawsze dodawane na końcu.



Typy indeksów - GIST

GiST (Generalized Search Tree)

- GiST to ogólna struktura indeksu, która może być używana do implementacji wielu różnych schematów indeksowania. Jest szczególnie przydatna dla typów danych, które nie są łatwo indeksowalne za pomocą B-tree, takich jak:
 - **Dane geometryczne:** Punkty, linie, poligony (np. z rozszerzeniem PostGIS).
 - **Dane pełnotekstowe:** Wyszukiwanie pełnotekstowe (np. z typem tsvector).
 - **Dane zakresowe:** WHERE range_column OVERLAPS range_value.
 - **Dane wielowymiarowe:** Indeksowanie danych w wielu wymiarach.



Plany zapytań

- PostgreSQL generuje plany zapytań w postaci drzewa kroków (nazywanych węzłami), które obrazują, jak optymalizator zamierza pobrać oraz przetworzyć dane dla danego zapytania.
- Analiza planu zapytania pozwala zidentyfikować wąskie gardła i podjąć kroki optymalizacyjne, aby usprawnić wykonanie SQL



Plany zapytań - struktura

- Plan zapytania w PostgreSQL to drzewo, w którym **każdy węzeł odpowiada operacji**, np. skanowaniu tabeli, odczytowi z indeksu, sortowaniu lub łączeniu danych (JOIN).
- Najniższe węzły drzewa to skany tabel (np. *Sequential Scan*, *Index Scan*, *Index Only Scan*), które zwracają surowe dane.
- Wyższe węzły realizują operacje pośrednie, takie jak łączenia (*Nested Loop*, *Merge Join*, *Hash Join*), filtrowanie, sortowanie czy agregacje (*Aggregate*).
- Każdy węzeł posiada własny koszt początkowy (*start-up cost*), koszt całkowity (*total cost*), prognozowaną liczbę zwróconych wierszy oraz szerokość rekordów

Plany
zapytań
-
kluczowe
elementy

- **Typy skanów:**
 - Sequential Scan – czytanie wszystkich wierszy tabeli, używane, gdy brak odpowiednich indeksów lub dla bardzo małych tabel.
 - Index Scan/Index Only Scan – korzystają z indeksów, preferowane przy większych tabelach i zapytaniach filtrujących po zindeksowanych kolumnach.
- **Joiny:**
 - Nested Loop – szybki dla małych ilości, ale nieefektywny przy dużych danych bez właściwych indeksów.
 - Hash Join/Merge Join – lepsze dla dużych zbiorów, przyklejone do odpowiednich warunków sortowania lub hash'owania.
- **Agregacje i sortowania**
 - kosztowne mogą być operacje GROUP BY, ORDER BY, DISTINCT, szczególnie na dużych zbiorach bez indeksów.



Plany zapytań - optymalizacja

- **Optymalizacja planów**
 - Upewnij się, że statystyki są aktualne (ANALYZE), bo plan powstaje dynamicznie na podstawie aktualnych danych.
 - Twórz odpowiednie indeksy na kolumnach filtrujących, sortujących bądź używanych do JOIN.
 - Analizuj koszt (cost) oraz liczbę przetwarzanych wierszy na poszczególnych krokach – wysokie wartości mogą wskazywać na potrzebę optymalizacji danego miejsca.
 - Unikaj nadmiarowych subzapytań – wielokrotne IN, EXISTS czy podzapytania mogą być mniej efektywne niż JOIN.
 - Zwracaj uwagę na operacje typowo kosztowne (np. sortowanie, materializacja pośrednia, skomplikowane agregaty).
 - W PostgreSQL 17 można także monitorować zużycie pamięci przez zapytanie oraz analizować wykorzystanie zapytań równoległych (parallel execution)

Plany zapytań - Analiza

```
EXPLAIN ANALYZE
WITH request AS MATERIALIZED (
SELECT id FROM customers ORDER BY id)

SELECT customer_id, COUNT(order_id)
FROM orders
WHERE customer_id IN (SELECT id FROM request)
GROUP BY customer_id
ORDER BY customer_id;
```

```
GroupAggregate (cost=2054.84..16699.38 rows=166709 width=12) (actual time=0.036..155.654 rows=100000 loops=1)
  Group Key: o.customer_id
```

CTE request

```
-> Index Only Scan using c_ix on customers c (cost=0.29..2052.29 rows=100000 width=4) (actual time=0.015..7.759
rows=100000 loops=1)
  Heap Fetches: 0
```

```
-> Merge Semi Join (cost=2.55..12146.46 rows=166709 width=8) (actual time=0.027..125.817 rows=300000 loops=1)
  Merge Cond: (o.customer_id = request.id)
```

```
-> Index Scan using co_ix on orders o (cost=0.57..4897548.71 rows=150100000 width=8) (actual time=0.007..61.174
rows=300001 loops=1)
```

```
-> CTE Scan on request (cost=0.00..2000.00 rows=100000 width=4) (actual time=0.017..24.345 rows=100000 loops=1)
```

```
Planning Time: 0.171 ms
Execution Time: 158.362 ms
```

- **CTE request (Index Only Scan)**
 - PostgreSQL korzysta z indeksu `c_ix` na tabeli `customers`. Wskazuje to, że odczyt ogranicza się do indeksu i nie wymaga fetchowania rekordów z dysku ("Heap Fetches: 0"), co jest bardzo wydajne. Liczba zwróconych wierszy jest zgodna z założeniem — 100 tys

Plany zapytań - Analiza

```
EXPLAIN ANALYZE
WITH request AS MATERIALIZED (
SELECT id FROM customers ORDER BY id)

SELECT customer_id, COUNT(order_id)
FROM orders
WHERE customer_id IN (SELECT id FROM request)
GROUP BY customer_id
ORDER BY customer_id;
```

```
GroupAggregate (cost=2054.84..16699.38 rows=166709 width=12) (actual time=0.036..155.654 rows=100000 loops=1)
  Group Key: o.customer_id
```

```
  CTE request
    -> Index Only Scan using c_ix on customers c (cost=0.29..2052.29 rows=100000 width=4) (actual time=0.015..7.759
rows=100000 loops=1)
      Heap Fetches: 0
```

```
    -> Merge Semi Join (cost=2.55..12146.46 rows=166709 width=8) (actual time=0.027..125.817 rows=300000 loops=1)
      Merge Cond: (o.customer_id = request.id)
```

```
        -> Index Scan using co_ix on orders o (cost=0.57..4897548.71 rows=150100000 width=8) (actual time=0.007..61.174
rows=300001 loops=1)
```

```
          -> CTE Scan on request (cost=0.00..2000.00 rows=100000 width=4) (actual time=0.017..24.345 rows=100000 loops=1)
```

```
Planning Time: 0.171 ms
Execution Time: 158.362 ms
```

- **Merge Semi Join**
 - Główna operacja łączenia (Merge Semi Join) odbywa się między tabelą orders a wynikiem CTE. Semi join jest używany dla warunku IN, pozwalając sprawnie porównać duże zbiory. Kluczowe jest, że obie tabele są posortowane i dołączone przez odpowiedni indeks, co umożliwia szybkie łączenie.

Plany zapytań - Analiza

```
EXPLAIN ANALYZE
WITH request AS MATERIALIZED (
SELECT id FROM customers ORDER BY id)

SELECT customer_id, COUNT(order_id)
FROM orders
WHERE customer_id IN (SELECT id FROM request)
GROUP BY customer_id
ORDER BY customer_id;
```

```
GroupAggregate (cost=2054.84..16699.38 rows=166709 width=12) (actual time=0.036..155.654 rows=100000 loops=1)
  Group Key: o.customer_id

  CTE request
    -> Index Only Scan using c_ix on customers c (cost=0.29..2052.29 rows=100000 width=4) (actual time=0.015..7.759
rows=100000 loops=1)
      Heap Fetches: 0

    -> Merge Semi Join (cost=2.55..12146.46 rows=166709 width=8) (actual time=0.027..125.817 rows=300000 loops=1)
      Merge Cond: (o.customer_id = request.id)

      -> Index Scan using co_ix on orders o (cost=0.57..4897548.71 rows=150100000 width=8) (actual time=0.007..61.174
rows=300001 loops=1)

      -> CTE Scan on request (cost=0.00..2000.00 rows=100000 width=4) (actual time=0.017..24.345 rows=100000 loops=1)

Planning Time: 0.171 ms
Execution Time: 158.362 ms
```

- **Index Scan on orders**
 - Tabela orders jest przeszukiwana za pomocą indeksu (co_ix). To szybka operacja, lepsza od pełnego skanowania sekwencyjnego, a liczba zwróconych wierszy (300 tys.) odpowiada warunkowi zapytania

Plany zapytań - Analiza

```
EXPLAIN ANALYZE
WITH request AS MATERIALIZED (
SELECT id FROM customers ORDER BY id)

SELECT customer_id, COUNT(order_id)
FROM orders
WHERE customer_id IN (SELECT id FROM request)
GROUP BY customer_id
ORDER BY customer_id;
```

```
GroupAggregate (cost=2054.84..16699.38 rows=166709 width=12) (actual time=0.036..155.654 rows=100000 loops=1)
  Group Key: o.customer_id

  CTE request
    -> Index Only Scan using c_ix on customers c (cost=0.29..2052.29 rows=100000 width=4) (actual time=0.015..7.759
rows=100000 loops=1)
      Heap Fetches: 0

    -> Merge Semi Join (cost=2.55..12146.46 rows=166709 width=8) (actual time=0.027..125.817 rows=300000 loops=1)
      Merge Cond: (o.customer_id = request.id)

      -> Index Scan using co_ix on orders o (cost=0.57..4897548.71 rows=150100000 width=8) (actual time=0.007..61.174
rows=300001 loops=1)

      -> CTE Scan on request (cost=0.00..2000.00 rows=100000 width=4) (actual time=0.017..24.345 rows=100000 loops=1)

Planning Time: 0.171 ms
Execution Time: 158.362 ms
```

- **GroupAggregate**
 - Połączenie wyników jest następnie agregowane (COUNT) wg customer_id, co jest optymalizowane przez wcześniejsze sortowanie wyników. Dodatkowo GROUP BY i ORDER BY uzyskują lepszą wydajność dzięki indeksom.

Plany zapytań - Analiza

```
EXPLAIN ANALYZE
WITH request AS MATERIALIZED (
SELECT id FROM customers ORDER BY id)

SELECT customer_id, COUNT(order_id)
FROM orders
WHERE customer_id IN (SELECT id FROM request)
GROUP BY customer_id
ORDER BY customer_id;
```

```
GroupAggregate (cost=2054.84..16699.38 rows=166709 width=12) (actual time=0.036..155.654 rows=100000 loops=1)
  Group Key: o.customer_id

  CTE request
    -> Index Only Scan using c_ix on customers c (cost=0.29..2052.29 rows=100000 width=4) (actual time=0.015..7.759
rows=100000 loops=1)
      Heap Fetches: 0

    -> Merge Semi Join (cost=2.55..12146.46 rows=166709 width=8) (actual time=0.027..125.817 rows=300000 loops=1)
      Merge Cond: (o.customer_id = request.id)

      -> Index Scan using co_ix on orders o (cost=0.57..4897548.71 rows=150100000 width=8) (actual time=0.007..61.174
rows=300001 loops=1)

      -> CTE Scan on request (cost=0.00..2000.00 rows=100000 width=4) (actual time=0.017..24.345 rows=100000 loops=1)

Planning Time: 0.171 ms
Execution Time: 158.362 ms
```

- **Czas wykonania**

- Planning Time oraz Execution Time pokazują rzeczywiste czasy wygenerowania i wykonania planu. Warto porównywać planowane (cost) i rzeczywiste (actual time) wartości, aby znaleźć rozbieżności i potencjalne miejsca wymagające optymalizacji

Plany
zapytań
-
na co zwracać
uwagę

- **Na co zwracać uwagę w analizie EXPLAIN**
 - **Różnica między szacowaną liczbą wierszy a rzeczywistą** — duże rozbieżności wpływają na wybór strategii skanów, JOINów, sortowań. Powtarzająca się niezgodność sugeruje potrzebę aktualizacji statystyk (ANALYZE).
 - **Typy skanów** — czy są preferowane Index/Index Only Scan zamiast Seq Scan przy dużych tabelach.
 - **Operacje kosztowne czasowo** — tam, gdzie "actual time" i "loops" są wysokie (np. JOIN lub GROUP BY), warto zastanowić się nad nowymi indeksami, optymalizacją SQL lub podziałem zapytania.
 - **Użycie CTE Materialized** — PostgreSQL 17 poprawił wydajność CTE, ale bez indeksacji w podzapytaniach może być kosztowny, jeśli nie jest efektywnie przetwarzany.



TOAST

- **TOAST (The Oversized-Attribute Storage Technique)** to mechanizm w PostgreSQL, który pozwala na efektywne przechowywanie bardzo dużych wartości pojedynczych kolumn (np. długich tekstów, dużych obiektów binarnych), które nie mieszczą się w standardowym rozmiarze strony danych (zwykle 8 kB)



TOAST

- **Jak działa TOAST?**
 - PostgreSQL ma ograniczenie, że pojedynczy wiersz danych nie może zajmować więcej niż rozmiar jednej strony (ok. 8 kB).
 - Jeśli wartość kolumny jest większa niż około 2 kB (parametr `TOAST_TUPLE_THRESHOLD`), mechanizm TOAST automatycznie:
 - próbuje skompresować dane,
 - jeśli to nie wystarczy, dzieli je na mniejsze kawałki (chunky),
 - te kawałki są przechowywane "na zewnątrz" w specjalnej, powiązanej z tabelą, tabeli TOAST.
 - W oryginalnej tabeli pozostaje tylko wskaźnik do tych fragmentów.



TOAST

- **Zastosowanie i korzyści**
 - Dzięki temu znacznie zmniejsza się rozmiar wiersza w głównej tabeli;
 - Zmniejsza liczbę operacji we/wy na dużych danych, bo dostęp do dużych pól jest leniwy i dokonuje się ich tylko, gdy są potrzebne;
 - Pozwala na przechowywanie bardzo dużych wartości typu text, bytea, jsonb, varchar(n), a także niektórych typów rozszerzonych;
 - Automatyczne, transparentne dla użytkownika.

TOAST jest ważnym elementem fizycznej architektury danych w PostgreSQL i jest domyślnie włączony dla większości typów danych o zmiennej długości, które mogą przekroczyć limit rozmiaru strony